M. Rohaan 221-2327 Sec-D

Q#01:-

Sol.

a) Condition

* Elements smaller than pivot → before partition index.
* Elements greater or equal to pivot → after partition index.
* Partition index = (count of smaller elements) - 1

```
Partition_A (arr, low, high):
{ pivot = arr(low)
  count = 0
  for i = low+1 to high:
      if arr[i] < pivot:
          count = count+1
  pIndex = low + count
  Swap ( arr [pIndex], arr [low])
  i = low
  j = high
```

```
while i< pIndex and j> pIndex
    if arr[i] < pivot
            i++
    else if arr[j] >= pivot
            j--
    else
      swap(arr[i], arr[j])
         i++
         j--
    return pIndex
}
```

b) * Elements smaller than pivot → before pivot
   * Elements greater or equal → after pivot

```
Partition_B(arr, low, high)
{ pivot = arr[high]
    i = low - 1
    for j = low to high - 1:
    if arr[j] < pivot:
      i = i+1
      swap(arr[i], arr[j])
    swap(arr[i+1], arr[high+1])
    return (i+1)
}
```

c)

```
Quick_Sort(arr, low, high)
    if low < high
        p = partition_A (arr, low, high)
Quick_Sort (arr, low, p-1)
Quick_Sort (arr, p+1, high)
```

CamScanner

Q#04:-

Sol.

```
Quick_Sort (A, low, high):
  While (low < high):
    pivot = Partition(A, low, high):
  //Recursively only on smaller side.
      if (pivot - low < high - pivot):
          Quick_Sort (A, low, pivot-1)
          low = pivot+1  //Tail call removed
                         ↳ handled by loop
      else
        Quick_Sort(A, pivot+1, high)
        high = pivot-1
```

Dry-Run Example

A = [5, 3, 8, 4, 2, 7, 1, 10]

1) low = 0, high = 7 Left subarray = [5, 3, 8, 4, 2, 7, 1] right empty

[(0, 7)] → [(0, 6)]

2) pivot = 1

array = [1, 3, 8, 4, 2, 7, 5, 10]

Left empty, right = [3, 8, 4, 2, 7, 5]

[(1, 6)]

CamScanner

3) pivot = 2

Array [1, 2, 8, 4, 3, 7, 5, 10]

Stack : [(2, 6)]

4) partition [8, 4, 3, 7, 5] pivot = 5

Array [1, 2, 4, 3, 5, 7, 8, 10]

Stack: [(2, 3), (5, 6)]

5) Final Sorted array

[1, 2, 3, 4, 5, 7, 8, 10]

Q#03:-

Sol.

if all elements are equal, then every partition in randomized quicksort will fail to divide the array because the pivot is equal to all other elements.

left side = empty

right side = remaining (n-1) elements

$T(n) = T(n-1) + O(n) = O(n^2)$

When all elements are equal, randomized quick sort runs in $O(n^2)$ time because each partition is unbalanced.

Q#05:-

Sol:-

a) A d-ary heap (like a binary heap but each node has upto d children) is stored in an array A[1...n] just like a binary heap.

For node at index i:

* parent(i) = $\lceil (i-2)/d \rceil + 1$
* kth child(i) = $d(i-1) + k + 1$ for $i \leq k \leq d$ and child(i) $\leq n$

e.g d=3 children of node 1 are at 2, 3, 4; children of node 2 are at 5, 6, 7, so on.

b) Height of a d-ary heap

Each level contains upto d^{th} nodes

So for n elements

$$n \simeq \frac{d^{h+1} - 1}{d - 1}$$

$$h = O(\log_d n)$$

CamScanner

c) Extract-Max for a d-ary heap

```
D_Ary_Extract_Max (A)
{ if A.heap-size < 1
      error "heap underflow"
  max = A[1]
  A[1] = A[A.heap_size]
  A.heap_size = A.heap_size - 1
  D_Ary_Max_Heapify (A,1)
  return max
}
```

D-Ary-Max-Heapify (A,i):

* find the largest among i and its d children
* swap A[i] with the largest child if needed
* Recursively call heapify on that child

Time Complexity

$O(d \cdot \log_d n)$

Q#07:-

Sol.

```
DualHeapSort (A)
{
  If (size(A)) ≤ 1 : return
  divide A into 2 parts → left, right
  build maxHeap(left)
  build minHeap(right)
  TreeSwap(maxHeapRoot, minHeapRoot)
  DualHeapSort(left)
  DualHeapSort(right)
}
```

Building heaps: $O(n)$

Each treeswap and recursively partioning:
$O(\log n)$ per level.

Total time: $O(n \log n)$

Space: $O(n)$

Q#09:-

Sol.

* Let total characters across all strings = n.
* Use Radix Sort (character by chracter) starting from the last character of each string.
* Use Counting Sort ($O(k+n)$) for each character position, where k is constant.

We can sort the strings in $O(n)$ time using Radix sort with Counting sort

Why $O(n)$:

* each character of every string is processed exactly once
* counting sort or characters takes $O(n)$ total since alphabets size (26) is constant. Hence total time $O(n)$

e.g [a, ab, b]

sort by last character → Then by previous ones → giving standard alphabetical order $a < ab < b$.

Q# 10:-

Sol.

a) Prof. Madni: $k = n^2$

* Expected items per bucket

$$m = \frac{n}{k} = \frac{n}{n^2} = \frac{1}{n} \text{ (mostly buckets empty)}$$

Time space is $O(n)$ but space is dominated by $O(n^2)$ which is poor choice, because sorting inside bucket is neglibile but initializing n^2 buckets takes $O(n^2)$ space

b) Ms. Sara: $k = \sqrt{n}$

* $m = \frac{n}{k}$ Expected items per bucket

$$= \frac{n}{\sqrt{n}} = \frac{\sqrt{n}\sqrt{n}}{\sqrt{n}} = \sqrt{n}$$

* Sorting cost per bucket: $O(m^2)$

$$= O((\sqrt{n})^2) = O(n)$$

* No. of buckets $= \sqrt{n}$

So total bucket sorting cost $\approx \sqrt{n} \cdot O(n)$

$$\approx O(n^{3/2})$$

* $O(n^{3/2})$ is worse choice because too few buckets make each bucket large

c) Max. sara : $k = \sqrt{n}$

Space Comparison

* Total space = $O(n) + O(k)$

$O(n) + O(\sqrt{n}) = O(n)$

$k = n$

$O(n) + O(n) = O(n)$

both use $O(n)$ space asymptotically i.e roughly the same total space.

Q#11:-

Sol.

Step #01:-

1) Find max value.
2) Create counting array.
3) Count frequency of each element.
4) Compute comulative sum.
5) Place elements in output array according to position.

Step#02:-

Counting sort uses array indices = element values so negative numbers can't be used directly.

Step#03:-

To handle negatives

* Find both min and max in input.
* Compute offset = -min.
* Shift all numbers by offset when sorting counts.

To sort in descending order, we'll

* Normal Counting sort gives ascending order.
* Reverse the final array or build it in reverse order.

CamScanner

Step # 04:

Modified Algorithm

Counting Sort with Negatives Descending (A)

1) Find min and max in A.
2) offset = -minval.
3) Create count array C [0.. (maxValue + offset)].
4) Initialize all C[i] = 0.
5) For each number x in A:
   C[x + offset]++.
6) Show C after counting.
7) For i=1 to length(C)-1. C[i] = C[i] + C[i-1]
8) Show C after cumulative sum.
9) Create output array B same size as A.
10) For each x in A (traverse normally for descending output):
   pos = (C[x+offset] - 1):
   B[pos] = x
   C[x + offset] --
11) Reverse B for descending order.
12) Return B

Step # 05:

Dry Run on [8, 5, -6, 7, 1, 4, 5]

1) min = -6
   max = 8

2) offset = -(-6) = 6

3) Create Count array C of size (8 + 6 + 1) = 15
   C[0..14] = 0
   Index Mapping;
   Value = Index - offset
         = Index - 6
   So index 0 → value = -6
      Index 6 → value = 0
      Index 14 → value = 8

4) Counting Occurences (input = [8, 5, -6, 7, 1, 4, 5])

Step	Value	Index = value + offset	Updated Carray
1	8	14	C[14] = 1
2	5	11	C[11] = 1
3	-6	0	C[0] = 1
4	7	13	C[13] = 1
5	1	7	C[7] = 1
6	4	10	C[10] = 1
7	5	11	C[11] = 2

[1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 2, 0, 1, 1]

CamScanner

5) Cumulative Sum (Prefix Sum)

i	C[i] before	C[i] after Cumulative Sum
0	1	1
1	0	1
2	0	1
3	0	1
4	0	1
5	0	1
6	0	2
7	1	2
8	0	2
9	0	3
10	1	5̄
11	2	5̄
12	0	6
13	1	7
14	1	

Final C after Cummulative Sum

[1,1,1,1,1,1,1,2,2,2,3,5̄,5̄,6,7]

6) Build output Array B (ascending first)

Process A = [8,5̄,-6,7,1,4,5̄] in normal order.

Step	Value	index = val + offset	C[index] before	Pos	C[index] after	B Partial result
1	8	14	7	6	6	[.....8]
2	5̄	11	5̄	4	4	[....5̄,..8]
3	-6	0	1	0	0	[-6,..5̄,...8]
4	7	13	6	5	5	[-6,..5̄,7,8]
5	1	7	2	1	1	[-6,1,..5̄,7,8]
6	4	10	3	2	2	[-6,1,4,5̄,7,8]
7	5̄	11	4	3	3	[-6,1,4,5̄,5̄,7,8]

CamScanner

B (descending order): [-6, 1, 4, 5, 5, 7, 8]

Step # 07:

[8, 7, 5, 5, 4, 1, -6]

CamScanner